

```
0000 ----- TLALOC--(The Lonesome Ada Loving Ol'timer Compiler) Strict Ada 83 C
ompiler-- Copyright (C) 2024-2026 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/
or modify-- it under the terms of the GNU General Public License as published by-- the Free Software Foundation, either version 3 of the License, o
r -- (at your option) any later version.-- This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without eve
n the implied warranty of-- MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the-- GNU General Public License for more details.-- You sh
ould have received a copy of the GNU General Public License-- along with this program. If not, see <https://www.gnu.org/licenses/>.--
----- Quelques explications sont n cessaires sur cette proc dure qui n'est pas tr s lo
ngue mais-- dont le fonctionnement par changement de signe des r f rences de page dans la version d'origine-- de Peregrine Systems n'est pas tr s cla
ir.-- Lors de l'analyse syntaxique (PAR_PHASE) des pages virtuelles d'arbre sont allou es pour-- l'arbre DIANA.-- Dans la phase librairie
o l'on charge les arbres d'unit s "withes", des pages-- suppl mentaires--ultrieures sont allou es, pages "withes" qui n seront pas sauvegard es--
0010 l'criture de l'unit  apr s analyse s mantique.-- La phase de v rification s mantique (SEM_PHASE) ajoute derri res les pages "withes" des pages-- qu
i devront, elles,  tre sauvegard es la phase WRITE_PHASE. Ainsi les pages "withes" coupent en deux-- la plage des pages sauvegard es-- Pages
AST --Pages withes | Pages SEM-- Une translation de recompaction avant sauvegarde est ainsi n cessaire et proc de en deux temps :-- On mar
que d'abord toutes les pages ne pas toucher dans la compaction (les pages au del -- de la derni re utilis e en SEM_PHASE) et toutes les pages d'unit s
withes-- On fait une recopie des pages de l'unit  sauvegard e afin de former une seule plage de-- pages sauvegard es dans le fichier de comp
ilation de l'unit --
MORIN--Universit  de Bretagne Occidentale--janvier 2025--
-----1-2-3-4-5-6-7-8-9-A-B-C-----with SEQUENTIAL_IO;separate( IDL)-----procedure-----WRITE_LIB-----is D
ONT_MOVE:= array( VPG_IDX ) of BOOLEAN;----- INDIQUE SI UNE PAGE N EST-PAS L OBJET DE DEPLACEMENTS DE TRANSLATION NEW_UNIT_SEQ:= SEQ_TYPE;-----
----- LA NOUVELLE LISTE D UNITES APRES COMPACTION----- procedure MARK_DONT_MOVE_PAGES( COMP_UNIT : TREE )-----
0020 K; begin----- TRANS_WITH_SEQ:= SEQ_TYPE:= LIST( COMP_UNIT );----- LISTE-FERMETURE-TRANSITIVE DES WITH HIGH_BLOCK:= PAGE_IDX:= LAST_BLOC
;-----TOUTES LES PAGES JUSQU A HIGH_BLOCK PEUVENT A PRIORI  TRE OBJET DE TRANSLATION DE COMPACTION-----LES AUTRES AU DESSUS N ONT PAS A
 TRE AFFECT ES----- DONT_MOVE( 1 .. HIGH_BLOCK ) := ( others=>FALSE ); DONT_MOVE( HIGH_BLOCK + 1 .. MAX_VPG ) := ( others=>TRUE );-----PAR
MI LES-PAGES SOUS HIGH_BLOCK, LES PAGES WITHES N ONT PAS A  TRE AFFECT ES PAR DE LA RECOPIE DE COMPACTION-----MARK_TRANS_WITH:
declare TRANS_WITH:= TREE; begin while not IS_EMPTY( TRANS_WITH_SEQ ) loop----- POUR TOUTE LA LISTE TRANSITIVE DES UNITES W
ITHES POP( TRANS_WITH_SEQ, TRANS_WITH );----- SORTIR UN POINTEUR DE BLOC INFO D UNIT  WITH E (AVEC ENT TE SPECIALE NON HI)-----
-----MARK_DONT_MOVE_PAGES_WITHES: declare WITHED_UNIT:= TREE:= D( TW_COMP_UNIT, TRANS_WITH );----- POINTEUR VERS N
OEUD UNIT -WITH E FIRST_PAGE:= PAGE_IDX:= WITHED_UNIT.PG; NBR_PAGES_WITHED:= PAGE_IDX;----- PAGE_IDX( DI( XD_NBR_PAGES, WITHED_UNIT ) ); be
gin NBR_PAGES_WITHED := PAGE_IDX( DI( XD_NBR_PAGES, WITHED_UNIT ) ); for I in FIRST_PAGE .. FIRST_PAGE + NBR_PAGES_WITHED - 1 loop DONT_MOVE(
I ) := TRUE;----- LES PAGES DES UNITES WITHES SONT DECLAREES HORS TRANSLATION end loop; end MARK_DONT_MOVE_PAGES_WITHES;
0030 end loop; end MARK_TRANS_WITH; end MARK_DONT_MOVE_PAGES;
procedure WRITE_UNIT( COMP_UNIT_ARG : TREE ) is package SEQ_IO is new SEQUENTIAL_IO( SECTOR ); COMP_UNIT:= TREE; LIB_FILE:= SEQ_IO
.FILE_TYPE;----- fonction RECOPIE POUR COMPACTION( T : TREE ) return TREE----- is begin
-----UN SOURCE_POSITION N EST PAS A RECOPIER-EN LUI MEME MAIS EXIGE DE RECOPIER LE SOURCE LINE-ASSOCIE----- if T.PT = S-then----- SO
URCE_POSITION return MAKE_SOURCE_POSITION(----- RECONSTRUIRE UNE POSITION-SOURCE-----RECOPIE POUR COMPACTION( GET_SOURCE_LINE( T ) ), GET
_SOURCE_COL( T ) );----- AVEC UN DN_SOURCELIN  TRANSLATE-----TOUS LES ELEMENTS TERMINAUX (PAS D ATTRIBUT PAS DE REF DE PAGE) SONT RENDUS TELS QU
ELS----- else if T.PT = HI or else T.PG = 0 then----- NUM_VAL COURTE OU NIL OU NON INITIALIS  OU TERMINAL STYLE DN_UNIVERSAL_INTEGER
return T;----- JUSTE-RETOURNER L ARGUMENT-----UN SYMBOL_REP N EST-PAS A RECOPIER EN LUI MEME MAIS EXIGE DE RECOPIER-LE TXT_REP ASSOC
IE----- else if T.TY = DN_SYMBOL_REP then----- POINTEUR A DN_SYMBOL_REP return RECOPIE_POUR_COMPACTION( D( XD_TEXT , T ) );-----
0040 MOVE( T.PG ) then----- PAGE A NE PAS TRANSLATER return T;----- NE RIEN FAIRE-----UN POINTEUR P/L FAISANT L OBJET DE REC
OPIE POUR COMPACTION----- else declare WORD_ZERO:= TREE:= DABS( 0 , T );----- ENT TE DU NOEUD-POINTE PAR T begin-----
-----CAS OU L ENT TE DU NOEUD DONNE POUR RECOPIE EST DENATUREE EN POINTEUR-L CE QUI INDIQUE UNE RECOPIE DEJA FAITE-DE CE NOEUD-----if WORD_ZERO.PT = L
then----- ENT TE DE NOEUD-DEJA RECOPIE CONTENANT UN POINTEUR L PG/LN AU REMPLACANT return ( P, TY=>WORD_ZERO.TY, PG=>WORD_ZERO.PG, LN=>
WORD_ZERO.LN );----- RETOURNER LE POINTEUR P RECONSTRUIT-AU REMPLAANT-----CAS OU L ENT TE DU NOEUD POINTE DONNE POUR RECOPIE EST NORMALE (HI VA
LU NSIZ)-----else----- CAS NORMAL D'UNE ENT TE DE NOEUD case T.TY is----- QUELQUES AJUSTEMENTS SUIVANT TYPE DE NOEUD when C
LASS_NON_TASK_NAME | DN_TASK_SPEC => D( XD_STUB , T, TREE_VOID ); D( XD_BODY , T, TREE_VOID ); when DN_INCOMPLETE => D( XD_FULL_TYPE_S
PEC , T, TREE_VOID ); when others => null; end case;-----RECOPIE ET MARQUAGE declare LENGTH:= WO
RD_ZERO.NSIZ; COPIED_T:= TREE:= MAKE( T.TY, LENGTH ); LPTR_TO_COPIED_T:= TREE:= ( L, TY=> COPIED_T.TY, PG=>COPIED_T.PG, LN=> COPIED_T.L
N ); INUTILE:= TREE; begin if T.TY = DN_NUM_VAL then----- POUR UNE VALEUR-NUMERIQUE DABS( 0, COPIED_T, DABS( 0, T ) );
0050 ----- NE PAS OUBLIER L ENT TE QUI CONTIENT LE SIGNE-DANS ABSS-!! end if; DABS( 0, T, LPTR_TO_COPIED_T );----- ENSUITE REMPLACE L'E
NT TE-DU RECOPIE PAR UN POINTEUR L AU TRANSLATE INDIQUE-QUE LA TRANSLATION A EU LIEU----- if T.TY = DN_TXTREP or T.TY = DN_NUM_VAL then----- T
EXTE-OU VALEUR-NUMERIQUE for I in 1 .. LENGTH loop----- RECOPIER LE CONTENU DABS( I, COPIED_T, DABS( I, T ) ); end loop
; else----- TOUS AUTRES NOEUDS----- if T.TY = DN_COMPILATION_UNIT then----- POUR UNE UNIT  DE COMPACTION INUTILE := RE
COPIE_POUR_COMPACTION( D( XD_LIB_NAME, T ) );----- TRANSLATER LE NOM EN LE CONVERTISSANT EN TXTREP (CONDUIT A DEUX RECOPIES AVEC CI-DESSOUS ! )
end if;-----PROPAGER RECOPIE: for I in 1 .. LENGTH loop----- POUR LES ATTRIBUTS DABS( I, COPIED_T, RECO
PIE_POUR_COMPACTION( DABS( I, T ) ) );----- METTRE LES ATTRIBUTS TRANSLATES end loop-PROPAGER RECOPIE; end if;
return COPIED_T; end RECOPIE ET MARQUAGE; end if; end; end-RECOPIE POUR COMPACTION;
----- begin COMP_UNIT := RECOPIE_POUR_COMPACTION ( COMP_UNIT_ARG );----- FAIRE-UNE RECOPIE COMPACT E-CREER LE FICHER UNIT :
declare SYM:= constant-TREE:= D( XD_LIB_NAME, COMP_UNIT );----- PRENDRE LE SYMBOLE DU NOM-DE L'UNIT  DANS LA LIBRAIRIE FILE_NAME:= const
ant-STRING:= IDL.LIB_PATH( 1..LIB_PATH_LENGTH ) & PRINT_NAME( SYM );----- CHAINE DU NOM PREFIX E begin SEQ_IO.CREATE( LIB_FILE, SEQ_IO.OUT
_FILE, FILE_NAME );----- CREER LE FICHER LIBRAIRIE end CREER LE FICHER UNIT ;-----PREPARER ECRITURE PAGES U
NITE: declare FIRST_PAGE:= VPG_IDX:= COMP_UNIT.PG;----- PREMIERE PAGE DE LA RECOPIE COMPACT E NBR_PAGES:= NATURAL( L
AST_BLOCK - FIRST_PAGE + 1 );----- NOMBRE DE PAGES-DE LA RECOPIE COMPACT E POINTER:= TREE:= ( P, TY=> DN_VOID, PG=> FIRST_PAGE, LN=> 0 );-----
FABRIQUER UN POINTEUR NON-TYPE POUR-TOUCHER LES PAGES INUTILE:= TREE; begin DI( XD_NBR_PAGES, COMP_UNIT, NBR_PAGES );----- PO
INTER LE NOMBRE DE PAGES-DANS LE NOEUD RACINE (DN_COMPILATION_UNIT) DE LA RECOPIE-----ECRIRE LES PAGES D'UNIT :----- DE
S OPERATIONS D'ECRITURE-A CONFINER DANS LE PAGE_MANAGER for I in 1 .. NBR_PAGES loop----- STOCKER LES PAGES INUTILE := DABS( 0,
POINTER );----- TOUCHER LA PAGE-POUR LA FORCER DANS-UNE PAGE PHYSIQUE SEQ_IO.WRITE( LIB_FILE, PAG ASSOC PAGE( POINTER.PG ) ).DATA.all
);----- ECRIRE LA PAGE POINTER.PG := POINTER.PG + 1;----- PAGE SUIVANTE end loop-ECRIRE LES PAGES D'UNIT ;
-----PREPARER ECRITURE PAGES UNIT ; SEQ_IO.CLOSE( LIB_FILE ); NEW_UNIT_SEQ := APPEND( NEW_UNIT_SE
Q, COMP_UNIT );----- AJOUTER A LA LISTE DES UNIT S end-WRITE_UNIT;-----begin OPEN_IDL_TREE_FILE( IDL.LIB_PATH( 1..LIB_PATH_LENGTH
) & "###.TMP" ); if DI( XD_ERR_COUNT, TREE_ROOT ) /= 0 then CLOSE_IDL_TREE_FILE; return; end if; NEW_UNIT_SEQ := ( TREE_NIL, TREE_NIL )
; declare USER_ROOT:= TREE:= D( XD_USER_ROOT, TREE_ROOT );----- RETIRER LA RACINE UTILISATEUR COMPACTION:= TREE:= D( XD_STRUCTURE,
USER_ROOT );----- EN EXTRAIRE LA COMPACTION COMP_UNIT_SEQ:= SEQ_TYPE:= LIST( D( AS_COMPLTN_UNIT_S, COMPACTION ) );----- PUIS LA LISTE D'UNI
TES DE-COMPACTION COMP_UNIT:= TREE; begin-----TRAITE LES UNIT S DE-COMPACTION while not IS_EMPTY(
COMP_UNIT_SEQ ) loop POP( COMP_UNIT_SEQ, COMP_UNIT ); if D( AS_ALL_DECL, COMP_UNIT ).TY = DN_VOID then----- UNIT -A PRAGMAS-SEULEM
ENT NEW_UNIT_SEQ := APPEND( NEW_UNIT_SEQ, COMP_UNIT );----- JUSTE-LA RECHAINER DANS LA LISTE MISE A JOUR else----- UNIT -U
SUELLE MARK_DONT_MOVE_PAGES( COMP_UNIT );----- NEW_BLOCK;----- FORCE-A SE METTRE AU DEBUT D'UN NOUVEAU BLOC POUR ALIGNER LA RECOPI
E COMPACT E WRITE_UNIT( COMP_UNIT );----- COMPACTER ET ECRIRE L UNIT  DE COMPACTION end if; end loop-TRAITE LES UNIT S DE-C
OMPACTION; LIST( D( AS_COMPLTN_UNIT_S, COMPACTION ), NEW_UNIT_SEQ );----- REMPLACER LA LISTE DES UNIT S
0080 PAR CELLE DES COMPACT ES, -SERA ECRIT A LA FERMETURE DU FICHER ARBRE end; DELETE_IDL_TREE_FILE;----- LE FICHER $$$TMP EST ABIM  PAR LE
PROCESSUS-DE MARQUAGE end-WRITE_LIB;
-----1-2-3-4-5-6-7-8-9-A-B-C-----
```